

[CS230] Shallow Neural Networks: Concept of Hidden Layer

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture - *Completed*

Chapter 2. Logistic Regression as a Neural Network - *Completed*

Chapter 3. Shallow Neural Networks (Current Unit)

- **3.1 Concept of Hidden Layer & Architecture**
- 3.2 Backpropagation Intuition
- 3.3 Random Initialization

Chapter 4. Deep Neural Networks - *Upcoming*

지난 시간 복습 및 연결

지금까지 우리는 로지스틱 회귀라는 '단일 뉴런(Single Neuron)'을 완벽하게 마스터했습니다. 하지만 뉴런 하나로는 단순한 선형 문제(직선으로 가르는 문제)밖에 해결하지 못합니다. 이제 이 뉴런들을 수직, 수평으로 연결하여 **진정한 의미의 신경망**을 구축할 시간입니다. 우리가 오늘 다룰 '얕은 신경망(Shallow Neural Network)'은 딥러닝이라는 거대한 마천루를 쌓기 위한 1층 기초 공사와 같습니다.

1 Unit Overview

□ 핵심 목표

이 단원은 로지스틱 회귀를 확장하여 **2-Layer** 신경망을 만드는 과정을 다룹니다.

- **개념:** 입력층과 출력층 사이에 있는 '은닉층(Hidden Layer)'의 역할과 정의를 이해합니다.
- **수학:** 층(Layer) 번호와 데이터 샘플 번호를 구분하는 표기법(Notation)을 익힙니다.
- **원리:** 왜 신경망에 **비선형 활성화 함수(Tanh, ReLU)**가 반드시 필요한지 증명합니다.
- **구현:** 행렬 연산을 통해 입력에서 출력까지 가는 **순전파(Forward Propagation)**를 구현합니다.

2 Essential Terminology

딥러닝 수학의 50%는 표기법을 제대로 아는 것에서 시작합니다.

표기	의미	예시 및 설명
$x = a^{[0]}$	입력층 (Input Layer)	원본 데이터. 가중치가 없으므로 0번째 층 취급.
$a^{[1]}$	은닉층 (Hidden Layer)	입력값을 변환하여 특징을 추출하는 중간 단계.
$a^{[2]} = \hat{y}$	출력층 (Output Layer)	최종 예측값 (예: 고양이일 확률).
$[l]$ (대괄호)	층 (Layer) 번호	$W^{[1]}$ (1번 층의 가중치)
(i) (소괄호)	데이터 샘플 번호	$x^{(i)}$ (i 번째 훈련 데이터)
$n^{[l]}$	l 번째 층의 뉴런 개수	$n^{[1]} = 4$ (은닉층 뉴런이 4개)

3 Core Concepts: 은닉층의 해부

3.1 1. Architecture (구조: 1층에서 2층으로)

로지스틱 회귀가 '입력 → 출력'의 직행버스라면, 얇은 신경망은 중간에 **환승 센터(은닉층)**가 하나 있는 구조입니다.

- **입력층 (Input):** x_1, x_2, x_3 (데이터 특성)
- **은닉층 (Hidden):** 입력 정보를 섞고 비틀어서 새로운 정보를 만듭니다.
- **출력층 (Output):** 은닉층의 정보를 종합하여 최종 결정을 내립니다.

□ 자동차 공장 조립 라인 (직관적 비유)

- **Input (x):** 철판, 유리, 고무 등 원자재.
- **Hidden Layer ($a^{[1]}$):** 공장 내부의 작업자들. 철판을 구부려 문짝을 만들고, 엔진을 조립합니다. 외부(사용자)에서는 이 과정이 보이지 않으므로 '**Hidden(은닉)**'이라고 합니다.
- **Output Layer ($a^{[2]}$):** 완성된 차를 검수하고 "출고 가능(1)" 혹은 "불량(0)" 판정을 내립니다.

3.2 2. 행렬 지옥 탈출 (Matrix Dimensions)

신경망 구현에서 가장 많이 틀리는 부분이 행렬의 크기 (Dimension)입니다. 아래 표를 보며 반드시 차원을 맞추는 연습을 해야 합니다.

- $n^{[0]} = n_x$: 입력 특성 개수 (예: 3)
- $n^{[1]}$: 은닉층 뉴런 개수 (예: 4)
- m : 데이터 개수 (예: 100)

변수	Shape (행, 열)	암기 공식
$W^{[1]}$	$(n^{[1]}, n^{[0]})$	(은닉 뉴런 수, 입력 특성 수)
$b^{[1]}$	$(n^{[1]}, 1)$	(은닉 뉴런 수, 1)
$Z^{[1]}, A^{[1]}$	$(n^{[1]}, m)$	(은닉 뉴런 수, 데이터 개수)
$W^{[2]}$	$(1, n^{[1]})$	(출력 뉴런 수, 은닉 뉴런 수)

3.3 3. 비선형성(Non-linearity)의 필요성

질문: "교수님, 그냥 계산하기 편하게 선형 함수($y = ax + b$)만 계속 쌓으면 안 되나요?"

답변: 절대 안 됩니다. 비선형 활성화 함수(Sigmoid, Tanh, ReLU)가 없다면 신경망은 깊어질 의미가 없습니다.

증명:

$$\text{Output} = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W'x + b'$$

선형 함수끼리의 결합은 결국 또 다른 하나의 선형 함수가 됩니다. 100층을 쌓아도 수학적으로는 1층짜리 로지스틱 회귀와 똑같아집니다. 복잡한 곡선을 그리려면 비선형 함수가 필수입니다.

4 Mathematical Forward Propagation

입력 x 가 신경망을 통과하는 과정을 수식으로 정리합니다.

Step 1: 입력 → 은닉층 (특징 추출)

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \tanh(Z^{[1]})$$

(참고: 은닉층에서는 *Sigmoid*보다 평균이 0인 *Tanh*가 학습 성능이 더 좋습니다.)

Step 2: 은닉층 → 출력층 (최종 예측)

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]}) \quad (\hat{y})$$

5 Practical Scenario: 얼굴 인식

□ 얼굴 인식 AI의 사고 과정 (실전 시나리오 & 계산)

- **Input (x):** 이미지의 각 픽셀 밝기값 (단순한 숫자 나열).
- **Hidden Layer ($a^{[1]}$):** 픽셀들을 조합하여 '선(Line)', '모서리(Edge)', '눈 모양', '코 모양' 같은 특징(**Feature**)을 찾아냅니다.
- **Output Layer ($a^{[2]}$):** 찾아낸 눈, 코, 입의 특징을 종합하여 "이것은 철수의 얼굴이다 (Probability)"라고 판단합니다.

6 Implementation (Python with NumPy)

```
1 import numpy as np
2
3 class ShallowNN:
4     def __init__(self, n_x, n_h, n_y):
5         np.random.seed(1)
6         # 가중치 초기화: 0이 아닌 작은 랜덤 값이어야 함! (Symmetry Breaking)
7         self.W1 = np.random.randn(n_h, n_x) * 0.01
8         self.b1 = np.zeros((n_h, 1))
9         self.W2 = np.random.randn(n_y, n_h) * 0.01
10        self.b2 = np.zeros((n_y, 1))
11
12    def forward(self, X):
13        """
14        X shape: (n_x, m)
```

```

15     """
16     # --- Layer 1 (Hidden) ---
17     # Z1: (n_h, m)
18     Z1 = np.dot(self.W1, X) + self.b1
19     A1 = np.tanh(Z1) # 은닉층 활성화 함수 (Tanh)
20
21     # --- Layer 2 (Output) ---
22     # Z2: (n_y, m)
23     Z2 = np.dot(self.W2, A1) + self.b2
24     A2 = 1 / (1 + np.exp(-Z2)) # 출력층 활성화 함수 (Sigmoid)
25
26     return A2
27
28 # --- 실행 예시 ---
29 if __name__ == "__main__":
30     # 개의3 특성, 개의4 데이터 샘플
31     X = np.array([[1, 2, 3, 4],
32                   [4, 5, 6, 7],
33                   [7, 8, 9, 10]]) # shape (3, 4)
34
35     # 입력(3) -> 은닉(4) -> 출력(1)
36     model = ShallowNN(n_x=3, n_h=4, n_y=1)
37     output = model.forward(X)
38
39     print("Output Shape:", output.shape) # (1, 4) 예상
40     print("Prediction:", output)

```

Listing 1: Shallow Neural Network Forward Propagation

7 FAQ: 초심자가 자주 묻는 질문

- **Q1. 가중치 W 를 왜 0으로 초기화하면 안 되나요?**
A. W 가 모두 0이면 은닉층의 모든 뉴런이 똑같은 계산을 하게 됩니다(대칭성 문제). 뉴런이 100개여도 사실상 1개인 것과 같습니다. 서로 다른 특징을 배우게 하려면 랜덤하게 깨뜨려야 (Break Symmetry) 합니다.
- **Q2. 은닉층 개수는 어떻게 정하나요?**
A. 하이퍼파라미터입니다. 정답은 없습니다. 문제의 복잡도에 따라 다르며, 실험을 통해 최적의 개수를 찾아야 합니다. 보통 입력 크기보다 약간 크게 잡는 것부터 시작합니다.

다음 단계 (Next Step)

이제 우리는 신경망의 뼈대를 세우고 신호(데이터)를 앞으로 보내는 법(Forward)을 알았습니다. 하지만 아직 학습은 하지 않았습니다. 다음 시간에는 예측값과 정답 사이의 오차를 구해서, 다시 뒤로 보내며 가중치를 수정하는 역전파(Backpropagation)에 대해 다룹니다. 이것이 딥러닝 학습의 진정한 핵심입니다.

□ 단원 요약 (Cheat Sheet)

1. **Hidden Layer:** 입력과 출력 사이에서 비선형적 특징을 추출하는 층.
2. **Notation:** $[l]$ 은 층 번호, (i) 는 데이터 번호. 혼동 금지!
3. **Non-linearity:** 활성화 함수 (Tanh, ReLU 등) 가 없으면 신경망은 단순 선형 회귀와 같다.
4. **Dimension:** $W^{[l]}$ 의 크기는 $(n^{[l]}, n^{[l-1]})$ 이다. (행렬 크기 주의)